

Informe de Diagnóstico

Fase 2 — Código base y catálogo de malos olores

Sistema de Préstamo de Equipos Tecnológicos

Dominio D3 — Laravel 13 + SQLite

Estudiante	roddytic
Docente	Victoria García Velásquez, MGP.
Repositorio	github.com/roddytic/prestamo-equipos
Entrega	Fase 2 — Fin de semana 7
Fecha	24 de julio de 2026

Metodología

Tras completar la implementación funcional mínima de los 3 casos de uso (Registrar Préstamo, Registrar Devolución, Consultar Historial de Préstamos) con su suite de 15 pruebas unitarias en verde, se revisó manualmente el código propio en `app/Services` y `app/Repositories` — el núcleo de la lógica de negocio — buscando los malos olores clásicos de Fowler (método largo, clase grande, duplicación, parámetros problemáticos, condicionales complejos). Deliberadamente **no se pulió el código antes de este diagnóstico**: la primera versión se escribió priorizando que los casos de uso funcionaran y las pruebas pasaran, tal como ocurre en una iteración real de desarrollo. Los 5 hallazgos siguientes son reales — no fueron insertados a propósito — y quedan documentados aquí para corregirse en la Fase 3 mediante refactorización, no antes.

#	Mal olor	Nivel (Fase 3)	Ubicación
1	Código duplicado entre repositorios	Clases y objetos	<code>UsuarioRepository.php</code> , <code>EquipoRepository.php</code>
2	Validación "buscar → lanzar excepción" duplicada	Métodos	<code>GestorPrestamos.php:37-47, 69-70</code>
3	Método con múltiples responsabilidades	Métodos	<code>GestorPrestamos.php:32-63</code>
4	Parámetro booleano (flag argument)	Condicionales	<code>GestorPrestamos.php:65, 73</code>
5	Data clump / obsesión primitiva	Datos	<code>GestorPrestamos.php:32</code>

1. Código duplicado entre repositorios

Duplicación

Nivel: Clases y objetos

Ubicación: `app/Repositories/UsuarioRepository.php:9-17` y `app/Repositories/EquipoRepository.php:9-17` (el mismo patrón se repite en `PrestamoRepository.php`).

```
9   public function buscar(int $id): ?Usuario
10  {
11      return Usuario::find($id);
12  }
13
14  public function guardar(Usuario $usuario): void
15  {
16      $usuario->save();
17  }
```

Por qué es un problema: `UsuarioRepository` y `EquipoRepository` son estructuralmente idénticos — solo cambia el nombre de la clase del modelo. Si mañana se decide agregar, por ejemplo, un método `existePor(string $campo)` común a los tres repositorios, habría que escribirlo y mantenerlo en tres archivos por separado. Es exactamente el escenario que Fowler describe como "si necesitas cambiar algo en más de un lugar, es una señal de diseño". Candidato para Fase 3: extraer una clase base genérica (`Repository`) de la que las tres hereden, o un método genérico apoyado en PHP templates/generics vía PHPDoc.

2. Validación "buscar → lanzar excepción" duplicada

Duplicación

Nivel: Métodos

Ubicación: `app/Services/GestorPrestamos.php:37-47` y `GestorPrestamos.php:69-70` .

```
37  if (! $usuario || ! $equipo) {
38      throw new OperacionInvalidaException('Usuario o equipo no encontrado.');
```

```
39  }
40
41  if (! $usuario->puedeSolicitarPrestamo()) {
42      throw new OperacionInvalidaException('El usuario no esta habilitado...');
```

```
43  }
44
45  if (! $equipo->estaDisponible()) {
46      throw new OperacionInvalidaException('El equipo no esta disponible.');
```

```
47  }
    ... (mismo patron: buscar → condicion → throw) reaparece en registrarDevolucion(),
    linea 69-70
```

Por qué es un problema: el patrón "busca una entidad, si no cumple una condición lanza `OperacionInvalidaException` con un mensaje" se repite con variaciones menores en `registrarPrestamo()` y `registrarDevolucion()` . No es código copy-paste idéntico, pero sí la misma *estructura* repetida — lo que Fowler llama duplicación "de esquema". Si se decide, por ejemplo, envolver estas excepciones con un código de error adicional, hay que recordar tocarlo en cada uno de los sitios donde aparece el patrón. Candidato para Fase 3: extraer un método privado de guarda, p. ej. `rechazarSi(bool $condicion, string $mensaje)` .

3. Método con múltiples responsabilidades: registrarPrestamo()

Método largo

Nivel: Métodos

Ubicación: `app/Services/GestorPrestamos.php:32-63` .

```
32 public function registrarPrestamo(int $usuarioId, int $equipoId, int $dias):  
    Prestamo  
33 {  
34     $usuario = $this->usuarios->buscar($usuarioId);  
35     $equipo = $this->equipos->buscar($equipoId);  
36  
37     if (! $usuario || ! $equipo) {  
38         throw new OperacionInvalidaException('Usuario o equipo no encontrado.');39     }  
40  
41     if (! $usuario->puedeSolicitarPrestamo()) { ... }  
45     if (! $equipo->estaDisponible()) { ... }  
49     $fechaPrestamo = Carbon::today();  
51     $prestamo = new Prestamo([ ...atributos... ]);  
58     $this->prestamos->guardar($prestamo);  
60     $equipo->cambiarEstado(EstadoEquipo::PRESTADO);  
62     return $prestamo;  
63 }
```

(Bloques abreviados con "..." para caber en el informe; el archivo real no tiene comentarios adicionales — ver

`GestorPrestamos.php:32-63` completo en el repositorio.)

Por qué es un problema: en 30 líneas, el método mezcla seis responsabilidades distintas (buscar, validar existencia, validar reglas de negocio, calcular fechas, construir/persistir, y mutar el estado de una entidad relacionada). Hoy es legible porque el caso de uso es simple, pero cada responsabilidad nueva (por ejemplo, una regla de negocio adicional) va a alargar el mismo método en vez de vivir en un lugar propio — el patrón clásico de crecimiento de un "long method". Candidato para Fase 3: Extract Method para separar la validación (`validarDisponibilidad(Usuario, Equipo)`) de la construcción/persistencia.

4. Parámetro booleano (flag argument) en registrarDevolucion()

Condición

Nivel: Condicionales

Ubicación: `app/Services/GestorPrestamos.php:65` (firma) y `línea 73` (uso).

```
65 public function registrarDevolucion(int $prestamoId, bool $danado = false):  
   Prestamo  
   ...  
73     $prestamo->equipo->cambiarEstado($danado ? EstadoEquipo::DANADO :  
     EstadoEquipo::DISPONIBLE);
```

Por qué es un problema: `$danado` es un parámetro booleano que cambia el comportamiento interno del método (qué estado final recibe el equipo). Robert C. Martin lo señala explícitamente en *Clean Code*: un parámetro bandera es indicio de que el método en realidad hace *dos cosas* ("registrar una devolución normal" y "registrar una devolución con equipo dañado") disfrazadas de una sola firma. Además, en el sitio de la llamada (`registrarDevolucion($id, true)`) el significado de `true` no es autoexplicativo sin mirar la firma del método. Candidato para Fase 3 (nivel condicionales): reemplazar el flag por dos métodos explícitos, o por un objeto de caso especial (`Replace Parameter with Explicit Methods` / `Introduce Special Case`).

5. Data clump / obsesión primitiva en los parámetros de registrarPrestamo()

Datos

Nivel: Datos

Ubicación: `app/Services/GestorPrestamos.php:32` .

```
32 public function registrarPrestamo(int $usuarioId, int $equipoId, int $dias):  
    Prestamo
```

Por qué es un problema: `$usuarioId` , `$equipoId` y `$dias` son tres primitivos sueltos que siempre viajan juntos para representar un mismo concepto: "una solicitud de préstamo". Es el "Data Clump" de Fowler — cuando los mismos datos aparecen agrupados una y otra vez, probablemente deberían ser su propio objeto. Hoy son solo 3 parámetros y parece inofensivo, pero si el caso de uso creciera (por ejemplo, agregando una fecha de inicio distinta a "hoy", u observaciones del encargado), este método terminaría con una lista de parámetros cada vez más larga en vez de un solo objeto de entrada. Candidato para Fase 3 (nivel datos): introducir un objeto de valor `SolicitudPrestamo` (Introduce Parameter Object).